

# **Blockchain-Based Library Book Lending Tracker**

Technical Report: Individual Assignment 1

**Student:** Joseph Nishimwe

**Date:** 25 September 2026

**GitHub:** [https://github.com/josep-prog/formative\\_introduction\\_blockchain](https://github.com/josep-prog/formative_introduction_blockchain)

**Demo video:** <https://youtu.be/YUA85d7KCcy0?si=26qMwWEon9HwG0Xh>

| File Name    | Link                         | Purpose of the file                                                                                                                                                             |
|--------------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DemoVideo    | <a href="#">DemoVideo</a>    | Full explanation of the project and a guide on how to run it.                                                                                                                   |
| Blockchain.c | <a href="#">blockchain.c</a> | Manages the blockchain: creates the genesis, borrow, return and overdue blocks, links them with hashes, finds active loans, validates the chain, and saves and loads chain.txt. |
| Blockchain.h | <a href="#">blockchain.h</a> | Defines the Block structure and declares the blockchain functions.                                                                                                              |
| Crypto.c     | <a href="#">crypto.c</a>     | Handles security with OpenSSL: makes the key pair, saves the encrypted private key and the public key, signs and verifies blocks, and hashes librarian PINs.                    |
| Crypto.h     | <a href="#">crypto.h</a>     | Declares the cryptographic functions used by main.c and blockchain.c.                                                                                                           |
| Main.c       | <a href="#">main.c</a>       | Controls the whole program: the login, the menu, and calling the other files to load                                                                                            |

|                |                                |                                                                                                           |
|----------------|--------------------------------|-----------------------------------------------------------------------------------------------------------|
|                |                                | records, create blocks and run security checks.                                                           |
| Registry.h     | <a href="#">registry.h</a>     | Defines the Book, Member and Librarian structures and declares the registry functions.                    |
| Registry.c     | <a href="#">registry.c</a>     | Loads books, members and librarians from the data files, reports bad lines, and finds a record by its ID. |
| Books.txt      | <a href="#">books.txt</a>      | The book registry: ID, title and author of each book.                                                     |
| Members.txt    | <a href="#">members.txt</a>    | The member registry: ID, full name and course code of each member.                                        |
| Librarians.txt | <a href="#">librarians.txt</a> | Staff who can log in: ID, name, role (ADMIN or LIBRARIAN) and a PBKDF2 hash of their PIN.                 |
| Makefile       | <a href="#">Makefile</a>       | Builds the whole program and links OpenSSL with one command (make).                                       |

## **1. Introduction**

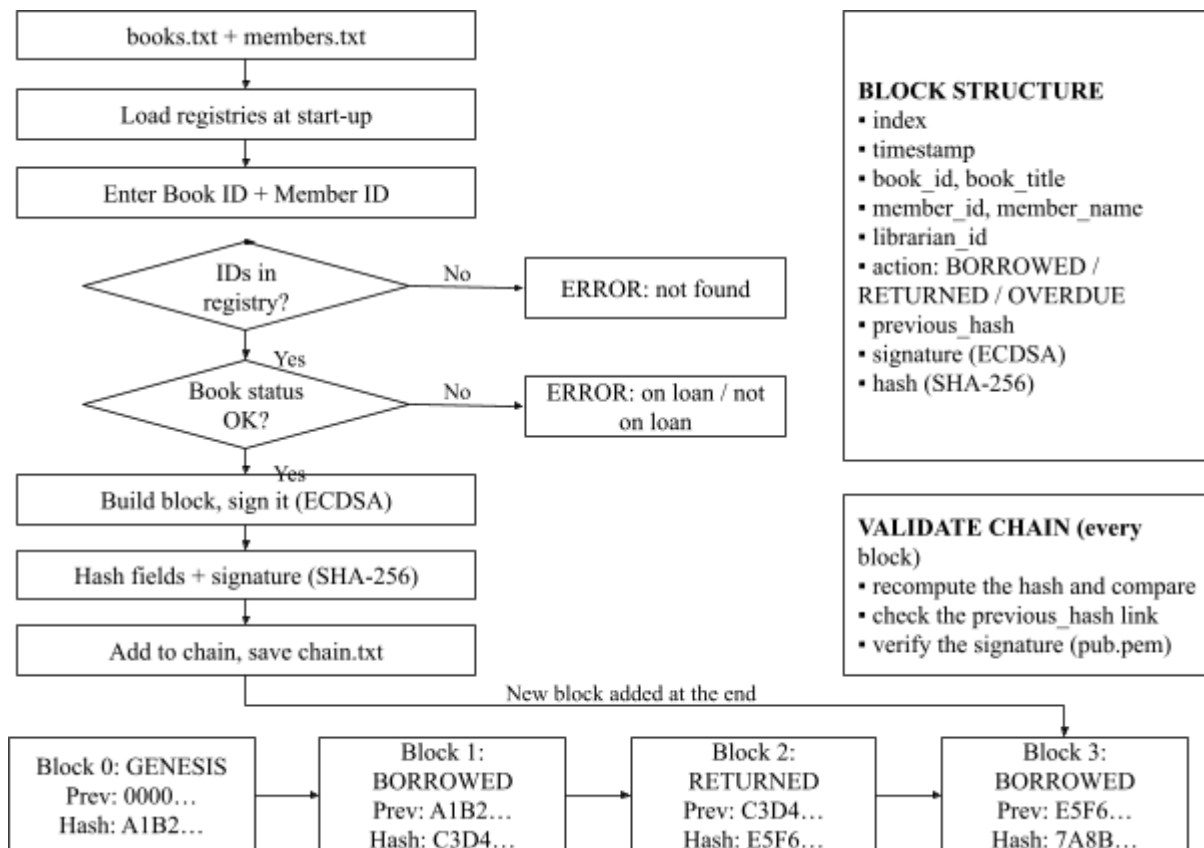
A library needs to know which books are out, who has them, and when they come back. Usually this is kept in a paper log or a normal database. The problem is that these records are easy to change. A librarian could mark a lost book as "returned", or a borrower could say the log is wrong, and nobody could prove what really happened.

In this project I built a small library lending system in the C language that solves this problem with a blockchain. Every time a book is borrowed or returned, the program writes a new record called a block. Each block is locked to the block before it with a SHA-256 hash, and each block is signed with a digital signature. Because of this, if anyone changes an old record, the program can detect it.

The program runs in the terminal. A librarian logs in, and then can borrow a book, return a book, view all records, check that the chain is still valid, mark overdue loans, and (as an admin) run a demonstration of tamper detection.

## Approach for Entire project

*System design: registry and lending flow, block structure, and chain linkage.*



I divided the program into different parts so that each part has a clear responsibility. This separation makes the program easier to understand because the code that deals with books and members is kept separate from the code that deals with the blockchain and cryptography.

## 2. Description of the Blockchain Implementation

### 2.1 How the code is organised

I split the program into four parts so that each part has one job. `main.c` shows the menu and connects everything. `registry.c` loads the books, members and librarians from text files and finds a record by its ID. `blockchain.c` creates blocks, links them together, checks the chain and saves it to a file. `crypto.c` does all the cryptography: making keys, signing,

checking signatures and hashing PINs. A Makefile builds everything with one command and links the OpenSSL library.

## 2.2 The book and member registries

When the program starts, it reads books.txt and members.txt into arrays of Book and Member structs. Each line has three fields separated by commas. For books these are the book ID, title and author. For members these are the member ID, full name and course code. If a file is missing or has no usable lines, the program prints an error and stops. A bad line (for example a missing field, a field that is too long, or a duplicate ID) is reported with its line number and skipped.

These registries are the first check. Before any block is created, the program looks up the book ID and the member ID. If either one is not in the registry, it prints ERROR: Book or Member not found and nothing is added to the chain. The match is exact, so an ID like BK001X is not accepted as BK001.

## 2.3 The block

Each lending event is stored in one block. The table below shows what is inside a block.

| Field                     | Type                  | What it holds                                                    |
|---------------------------|-----------------------|------------------------------------------------------------------|
| index                     | int                   | Position of the block in the chain. The first block is 0.        |
| timestamp                 | time_t                | The time the block was created.                                  |
| book_id, book_title       | char[20],<br>char[80] | The book. The title is copied from the registry at that moment.  |
| member_id,<br>member_name | char[20],<br>char[50] | The member. The name is copied from the registry at that moment. |
| librarian_id              | char[20]              | The librarian who recorded the action (my own addition).         |
| action                    | char[10]              | BORROWED, RETURNED or OVERDUE (GENESIS for block 0).             |
| previous_hash             | char[65]              | The hash of the block before this one.                           |
| signature                 | unsigned<br>char[72]  | ECDSA digital signature of the block data, with its length.      |
| hash                      | char[65]              | SHA-256 hash of all the fields above, including the signature.   |

I copy the book title and member name into the block instead of only keeping the IDs. This way the block remembers exactly what was true at the time, even if the registry file changes later.

## **2.4 The genesis block and the chain**

The chain always starts with a genesis block. It has index 0, the action GENESIS, and a previous hash made of 64 zeros, because there is no block before it. Every block after that stores the hash of the block before it in its `previous_hash` field. This is what makes it a chain: Genesis, then Borrow, then Return, then Borrow, and so on. If one old block changes, its hash changes, and the next block no longer points to it correctly.

## **2.5 Borrowing a book**

The librarian types a book ID and a member ID. The program checks both IDs in the registries. Then it checks if the book is already on loan. It does this by reading the chain from the newest block backwards and finding the last record for that book. If the last record is BORROWED (or OVERDUE), the book is still out and the program refuses. If the book is free, the program builds a new BORROWED block, signs it, calculates its hash, adds it to the end of the chain and saves the chain to the file.

## **2.6 Returning a book**

For a return, the program again checks both IDs. It then looks for the book's active BORROWED block. If there is none, because the book was never borrowed or was already returned, it prints an error. I also added one more rule: the member returning the book must be the member who borrowed it. If everything is correct, a RETURNED block is created, signed, hashed, added and saved.

## **2.7 Overdue loans**

Menu option 5 looks at every book. If its last record is BORROWED and it is older than the loan period (14 days by default), the program adds an OVERDUE block. The book stays on loan until it is returned. A loan is only marked overdue once.

## **2.8 Validating the chain**

The function `validate_chain()` answers one question: has anything been changed? It goes through every block and checks that (1) the index matches the position, (2) the stored hash is the same as a freshly calculated hash, (3) the `previous_hash` matches the hash of the block before, and (4) the digital signature is valid. It also checks that block 0 is a real genesis block. If any check fails, the program prints which block failed and why.

## 2.9 Tamper detection

Option 6 shows tamper detection. It changes the book title in block 1 to "TAMPERED TITLE" in memory, without updating the hash or signature, and then validates the chain. The validation fails and reports block 1. The change is only made in memory, and the program never saves a chain that fails validation, so the file on disk stays clean. Tampering can also be shown by editing data/chain.txt in a text editor. The next time the program starts, or when `./library --verify` is run, it reports that the chain is invalid.

## 3. Security Mechanisms

### 3.1 SHA-256 hashing

A hash is like a fingerprint for data. SHA-256 turns any data into a 64-character code. If even one letter of the data changes, the code becomes completely different. In my program, all fields of a block are joined together in a fixed order, and SHA-256 is calculated over them and the signature. This hash is stored in the block, and the next block stores it as its previous hash. This is how the blocks are linked and how changes are detected.

### 3.2 Digital signatures (ECDSA)

Hashing shows that data has changed, but someone who changes a block could also calculate a new hash. Digital signatures stop this. The first time the program runs, it creates a key pair on the P-256 curve using OpenSSL. The private key signs each lending block. The public key checks the signature. Without the private key, nobody can make a valid signature for a changed block, so a rewritten block is caught even if its hash was updated.

The private key is saved in data/key.pem. It is encrypted with AES-256 using a passphrase, and the file can only be read by its owner. So a copy of the file alone is not enough to sign blocks. The public key is saved separately, without encryption, in data/pub.pem. The program uses only this public key when it checks signatures. This means checking the records never needs the secret key. Anyone can run `./library --verify` to check the whole

chain without a passphrase or login. When the program starts, it also checks that `pub.pem` really belongs to the signing key, and stops if it does not.

Before the menu appears, a librarian must log in with an ID and a PIN. The PINs are never stored as plain text. The file `librarians.txt` stores a PBKDF2-HMAC-SHA256 hash of each PIN, calculated 100,000 times, which makes guessing slow. The PIN is hidden while it is typed, and the comparison is done in constant time. After three wrong tries, the program closes. Each librarian has a role. A LIBRARIAN can borrow, return, view, validate and mark overdue loans. Only an ADMIN can run the tamper demonstration. The ID of the logged-in librarian is written into every block and is covered by the signature.

All data is stored in plain text files in the data folder. The registries (`books.txt`, `members.txt`, `librarians.txt`) are read at start-up. The blockchain is stored in `chain.txt`, with one block per line and the fields separated by the `|` character. The signature is written as hexadecimal text.

The chain is saved after every borrow, return or overdue action, so no records are lost when the program closes. To avoid a broken file if the program crashes while saving, it first writes to a temporary file and then renames it over the real file. Renaming is a single step, so the file is either the old version or the new version, never half written. When the program starts, it loads `chain.txt` and validates it straight away. If the file does not exist, a new chain with a genesis block is created. The keys are also kept between runs, so old signatures can still be checked.

## 5. Error Handling Strategy

My approach is simple: check everything before changing anything, and never leave the chain in a bad state. The table below shows the main errors and what the program does.

| Situation                                                           | What the program does                                      |
|---------------------------------------------------------------------|------------------------------------------------------------|
| <code>books.txt</code> or <code>members.txt</code> missing or empty | Prints an error and stops.                                 |
| A bad line in a registry file                                       | Prints a warning with the line number and skips the line.  |
| Unknown book ID or member ID                                        | Prints ERROR: Book or Member not found. No block is added. |
| Borrowing a book that is already out                                | Prints an error. No block is added.                        |

| Situation                                                    | What the program does                                                            |
|--------------------------------------------------------------|----------------------------------------------------------------------------------|
| Returning a book that is not on loan, or by the wrong member | Prints an error. No block is added.                                              |
| Wrong passphrase or wrong PIN                                | Prints an error. After three wrong PINs, access is denied.                       |
| pub.pem does not match the signing key                       | Prints an error and stops.                                                       |
| chain.txt cannot be read                                     | Prints an error and stops, so a damaged file is not overwritten.                 |
| chain.txt was tampered with                                  | Prints a warning with the bad block and reason. An invalid chain is never saved. |
| Letters typed where a number is expected                     | Prints "Invalid choice" and shows the menu again.                                |
| Input closed (for example Ctrl+D)                            | Exits cleanly instead of looping.                                                |
| Chain reaches its maximum size                               | Prints an error and refuses new blocks.                                          |

Input is always read one full line at a time, and every text field is checked against its maximum size before it is copied. This prevents long input from overflowing a buffer or spilling into the next question.

## 6. Screenshots of Application Execution

The screenshots below show the program running.

```
joe@code:~/Documents/formative_introduction_blockchain/formative_introduction_blockchain$ ./library
Loaded 5 books, 5 members and 2 librarians.
Key passphrase:
Digital signing key loaded from data/key.pem.
Loaded 1 blocks from data/chain.txt.
```

*Program start-up. Registries loaded, signing key unlocked, chain loaded or created.*

```
Librarian ID: LIB001
PIN:
Welcome, Joseph Nishimwe (ADMIN).

LIBRARY MENU
1. Borrow a book
2. Return a book
3. View all lending records
4. Validate the blockchain
5. Mark overdue loans
6. Tamper-detection demo (ADMIN)
7. Exit
```

*Librarian login and the main menu.*

```
Choice: 1
Book ID : BK001
Member ID: ALU002
Borrowed 'The Money Trap: Lost Illusions Inside the Tech Bubble' for Joseph Habimana.
```

*Borrowing a book with a valid book ID and member ID.*

```
Choice: 1
Book ID : BK002
Member ID: ALU999
ERROR: Book or Member not found
```

*Invalid book or member ID rejected ("ERROR: Book or Member not found").*

```
Choice: 3
-----
Block #0 [GENESIS]
  Time   : Fri Sep 25 16:15:56 2026
  Hash   : 9a94ec9c9b814b74776be39298aaffbc8d0081bdb117c23663f66c0146028439
  Prev   : 0000000000000000000000000000000000000000000000000000000000000000
  Signature: n/a (genesis block)
-----
Block #1 [BORROWED]
  Book    : The Money Trap: Lost Illusions Inside the Tech Bubble (BK001)
  Member  : Joseph Habimana (ALU002)
  By      : LIB001
  Time    : Fri Sep 25 16:19:07 2026
  Hash    : 608084111dc16c0e885e1486e2d9c85b27d1994d59619ae8972a785de039f81b
  Prev    : 9a94ec9c9b814b74776be39298aaffbc8d0081bdb117c23663f66c0146028439
  Signature: VALID (72 bytes)
```

*Viewing all lending records with signature status.*

```
Choice: 4
Blockchain is VALID - no tampering detected.
```

*Validating the chain ("Blockchain is VALID").*

```
Choice: 6
Changing Block #1's book title in memory only, without re-hashing or re-signing.
Blockchain is INVALID - tampering detected!
  Block #1: stored hash does not match the block's contents.
(Restart the program to reload the clean chain from disk.)
```

*Tamper detection demo ("Blockchain is INVALID - tampering detected!").*

```
joe@code:~/Documents/formative_introduction_blockchain/formative_introduction_blockchain$ ./library --verify
Checked 2 blocks from data/chain.txt with data/pub.pem.
Blockchain is VALID - no tampering detected.
```

*Checking the chain with only the public key using ./library --verify.*

```
joe@code:~/Documents/formative_introduction_blockchain/formative_introduction_blockchain$ mv data/members.txt data/members.backup
joe@code:~/Documents/formative_introduction_blockchain/formative_introduction_blockchain$ ./library
ERROR: Could not open 'data/members.txt'. Member registry is required to start the system.
```

*Error when books.txt or members.txt is missing or empty.*

```
joe@code:~/Documents/formative_introduction_blockchain/formative_introduction_blockchain$ ./library
Loaded 5 books, 5 members and 2 librarians.
Key passphrase:
ERROR: Could not open data/key.pem - wrong passphrase or damaged key file.
```

*Error when the key passphrase is wrong, so the signing key cannot be unlocked.*

## 7. Challenges Encountered and Solutions

---

**Validation was changing the data it checked.** At first, to check a block, I recalculated its hash directly on the block. This overwrote the stored hash, so a changed block could look correct. I fixed this by recalculating the hash on a copy of the block and comparing the copy with the original. Now validation only reads the chain and never changes it.

**Signatures do not always have the same length.** An ECDSA signature is usually 70 to 72 bytes, not a fixed size. If the program always read 72 bytes, it would check extra bytes that are not part of the signature, and verification would fail. I solved this by storing the real length of each signature next to it, and by refusing any signature longer than 72 bytes.

**Records and keys were lost between runs.** In my first version, the chain was only kept in memory and a new key was made every time the program started. When the program closed, all records were gone, and old signatures could not be checked with the new key. I now save the chain to chain.txt after every action, and I save the key to a file and load it again on the next run. To protect it, the file is encrypted with a passphrase and only the owner can read it.

**Checking the chain needed the secret key.** The program first took the public key from the encrypted private key file, so nobody could check the records without the passphrase. I fixed this by saving the public key in its own file, pub.pem, and using only that file for checking. I also added ./library --verify so anyone can check the chain.

**A crash while saving could break the chain file.** If the program stopped in the middle of writing chain.txt, the file would be half written. I solved this by writing to a temporary file first and then renaming it, which happens in one step.

**The tamper demo could damage the real records.** Once I added saving, there was a new risk: after the demo changed a block, the next save would have written the tampered chain to disk. I solved this by making the program validate the chain before every save and refuse to save an invalid chain. The demo only changes memory, and restarting the program reloads the clean chain.

**Special characters in the data files.** The chain file uses | to separate fields, so a title containing | would break it. The registry loader now rejects such lines. It also removes extra spaces and handles Windows line endings.

## 8. System Design Diagram

The diagram shows three things. First, how the registries are loaded at start-up and used to check every book ID and member ID before a block is made. Second, how each block points to the block before it through its `previous_hash`, starting from the genesis block. Third, what is inside one block. It also shows the steps of the lending flow (check IDs, check the book's history, build, sign, hash, add, save) and the three checks done when the chain is validated.

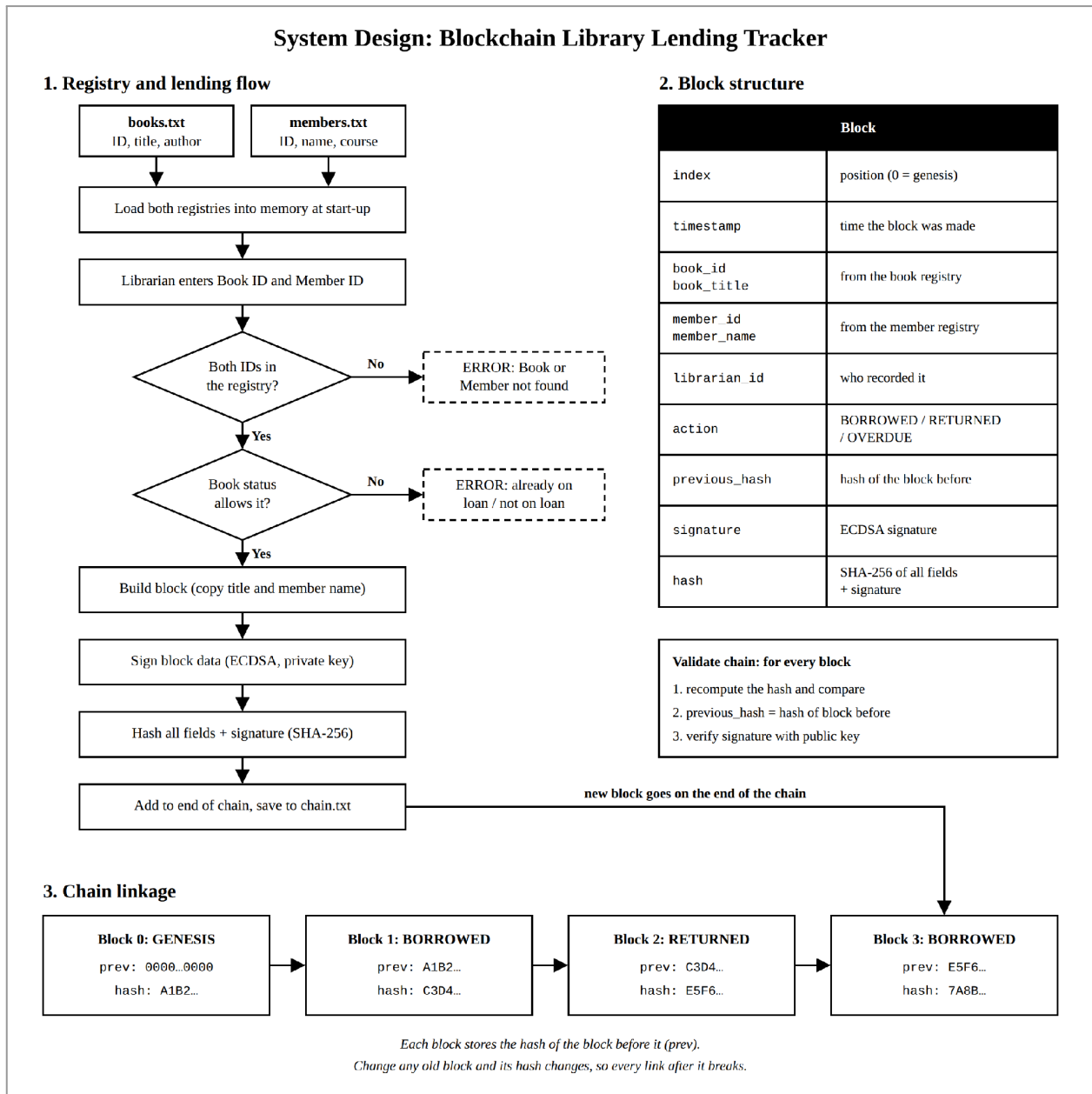


Figure 1: System design: registries, lending flow, chain linkage and block structure.

## 9. Conclusion

---

This project shows how a blockchain can make library records trustworthy. The registries decide which books and members are valid. The blockchain keeps a full history that is never overwritten, only added to. SHA-256 links the blocks so that any change is visible, and digital signatures prove that each block was made by the system and by a logged-in librarian. The validation step can check the whole history at any time, with only the public key. The system is small, but it shows the main ideas of a blockchain clearly: blocks, hashing, linking, signing and tamper detection.